Labo 16 — Labo pratique : budget, diagnostic, droits et cloisons

Objectif : déployer l'API avec des ressources déclarées, lire ce que le nœud en fait, provoquer un `OOMKilled` et un `Pending`, poser un LimitRange et une quota, redémarrer un conteneur et lire ses logs précédents, déboguer sans shell dans l'image, donner à un compte de service juste le droit de lire, puis fermer le réseau de l'API à tout sauf au front.

Prérequis — Labo 15 terminé : k3s, l'image `localhost/api:1.0` importée dans containerd (labo 12).

Fichiers fournis (`files/`) - `deployment-api.yaml` — l'API avec `requests` / `limits` et son Service. - `pod-oom.yaml` — un Pod qui alloue 100 Mo avec une limit de 32 Mi. - `pod-greedy.yaml` — un Pod qui demande 64 Gi. - `limitrange.yaml` — un LimitRange et une ResourceQuota pour le namespace. - `rbac-reader.yaml` — ServiceAccount `reader`, Role `pod-reader`, RoleBinding. - `netpol-api.yaml` — NetworkPolicy : seuls les Pods `app=web` joignent l'API.

Étape 1 — Requests, limits et ce qu'en fait le nœud

```
mkdir -p ~/labo-docker/16 && cd ~/labo-docker/16 && cp <chemin-du-labo>/files/* .
kubectl create namespace lab16 ; kubectl config set-context --current --namespace=lab16
kubectl apply -f deployment-api.yaml ; kubectl rollout status deployment/api --timeout=60s
kubectl get pods -o custom-
columns=NAME:.metadata.name,QOS:.status.qosClass,REQ:.spec.containers[0].resources.requests.mem
ory,LIM:.spec.containers[0].resources.limits.memory
kubectl describe node | sed -n '/Allocated resources/,/Events/p' | head -9
```

Observez deux Pods `Burstable` avec `64Mi` / `256Mi` ; et dans le nœud, `cpu 400m (2%)` de requests et `memory 268Mi (0%)` : vos deux Pods (200m, 128Mi) **plus** ceux de `kube-system`.

```
sleep 30 ; kubectl top nodes ; kubectl top pods
```

Observez `wsl-ubuntu 35m 0% 789Mi 2%` (vos valeurs diffèrent) et, pour chaque Pod de l'API, quelques millicœurs et `22Mi` de mémoire réelle : bien sous la request de 64 Mi, très loin de la limit. Si vous lisez `metrics not available yet`, attendez encore 30 s : metrics-server échantillonne toutes les 15 s.

Explication. Le tableau `Allocated resources` compte les **requests** (ce que le scheduler a réservé) ; `kubectl top` montre la **consommation réelle**. Les deux ne coïncident jamais, et c'est l'écart entre eux qu'on règle.

Étape 2 — `OOMKilled` et `Pending`

```
kubectl apply -f pod-oom.yaml ; sleep 15 ; kubectl get pod glouton
kubectl describe pod glouton | grep -E 'Reason:|Exit Code|Restart Count'
kubectl get pod glouton -o
jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}' ; echo
```

Observez `glouton 0/1 OOMKilled 1 (14s ago)` (ou déjà `CrashLoopBackOff`) ; `Reason: OOMKilled` et `Exit Code: 137` **deux fois** (l'état courant et l'état précédent : le conteneur relancé est mort de la même façon), `Restart Count: 1` ; et `OOMKilled` dans `lastState`.

```
kubectl apply -f pod-greedy.yaml ; sleep 5 ; kubectl get pod trop-gourmand
kubectl describe pod trop-gourmand | grep FailedScheduling | cut -c1-150
kubectl delete pod glouton trop-gourmand --now
```

Observez `trop-gourmand 0/1 Pending` et l'événement `Warning FailedScheduling … 0/1 nodes are available: 1 Insufficient memory`.

Explication. La limit est appliquée par le cgroup : 100 Mo dans 32 Mi, le noyau tue, le kubelet relance. La request est appliquée par le scheduler : 64 Gi sur un nœud qui ne les a pas, personne ne place, rien ne démarre.

Étape 3 — LimitRange et ResourceQuota

```
kubectl apply -f limitrange.yaml
kubectl run sans-limites --image=docker.io/library/alpine --command -- sleep 3600 ; sleep 2
kubectl get pod sans-limites -o jsonpath='{.spec.containers[0].resources}' ; echo
kubectl describe quota budget | sed -n '/Resource/,$p'
```

Observez un Pod créé sans `resources` qui en a pourtant : `{"limits":{"cpu":"250m","memory":"128Mi"},"requests":{"cpu":"50m","memory":"32Mi"}}` ; et la quota : `limits.memory 640Mi / 2Gi`, `pods 3 / 10`, `requests.cpu 250m / 2`, `requests.memory 160Mi / 1Gi`.

```
kubectl run enorme --image=docker.io/library/alpine --overrides='{"spec":{"containers":[{"name":"e","image":"docker.io/library/alpine","command":["sleep","1"],"resources":{"limits":{"memory":"4Gi"}}}]}}' 2>&1 | cut -c1-160
kubectl delete pod sans-limites --now
```

Observez `Error from server (Forbidden): pods "enorme" is forbidden: maximum memory usage per Container is 1Gi, but limit is 4Gi`.

Explication. Le LimitRange **complète** les Pods sans ressources (défauts) et **refuse** ceux qui dépassent `max` ; la quota compte tout ce qui vit dans le namespace. Les deux agissent à la création, jamais après.

Étape 4 — Événements, redémarrage, `logs --previous`, `debug`

```
kubectl get events --sort-by=.lastTimestamp | tail -5 | cut -c1-120
POD=$(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}')
kubectl exec $POD -- sh -c 'kill 1' ; sleep 5 ; kubectl get pod $POD
kubectl logs $POD --previous | tail -1
```

Observez les derniers événements du namespace (`Killing`, `Scheduled`, `Pulled`, `Started`…) ; après `kill 1`, le même Pod avec `RESTARTS 1` ; et dans les logs **précédents**, `API arretee proprement.` : la dernière ligne du conteneur mort.

```
kubectl debug $POD -it --image=docker.io/library/alpine --target=api --profile=general -- sh -c
'ps aux | head -3; wget -qO- localhost:8080/actuator/health'
```

Observez après deux lignes d'information de `kubectl` : `1 1000 … java -jar /app/api.jar` dans `ps` — le processus de l'API, vu depuis un conteneur `alpine` qui n'est pas dans l'image — puis `{"status":"UP"}`.

Explication. Le Pod a survécu, le conteneur a été remplacé : `logs` sans option montre le nouveau, `--previous` l'ancien. `kubectl debug --target` prête un shell et des outils au conteneur sans en mettre dans l'image.

Étape 5 — RBAC : un compte qui ne peut que lire

```
kubectl apply -f rbac-reader.yaml
kubectl auth can-i get pods      --as=system:serviceaccount:lab16:reader
kubectl auth can-i delete pods   --as=system:serviceaccount:lab16:reader
kubectl auth can-i get secrets   --as=system:serviceaccount:lab16:reader
kubectl auth can-i get pods -n default --as=system:serviceaccount:lab16:reader
```

Observez `yes`, `no`, `no`, `no` : lire les Pods d'ici, oui ; les supprimer, voir les Secrets, ou lire ailleurs, non.

```
kubectl get pods --as=system:serviceaccount:lab16:reader | head -2
kubectl get secrets --as=system:serviceaccount:lab16:reader 2>&1 | cut -c1-140
kubectl auth can-i --list --as=system:serviceaccount:lab16:reader | head -6
```

Observez la liste des Pods qui passe ; puis `Error from server (Forbidden): secrets is forbidden: User "system:serviceaccount:lab16:reader" cannot list resource "secrets"…` ; et dans `--list`, `pods/log` et `pods` avec `[get list watch]`, plus quelques droits génériques (`selfsubjectreviews`, `/api`…) que tout sujet authentifié possède.

Explication. Le Role dit **quoi** (`pods`, `pods/log` ; `get list watch`), le RoleBinding dit **qui** (`reader`), le namespace borne le tout. `--as` demande à l'API server de répondre « comme si » : pas besoin de kubeconfig pour tester.

Étape 6 — NetworkPolicy : fermer l'API

```
kubectl run client --image=docker.io/library/alpine --restart=Never --command -- sleep 600 ;
sleep 5
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health ; echo
kubectl apply -f netpol-api.yaml ; sleep 5
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health ; echo "exit=$?"
```

Observez `{"status":"UP"}` avant la politique ; après, `wget: can't connect to remote host (10.43.x.x): Connection refused` et `exit=1`.

```
kubectl label pod client app=web ; sleep 3
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health ; echo
kubectl get networkpolicy
```

Observez `{"status":"UP"}` de nouveau, sans rien redéployer : le label suffit ; et `api-only-from-web app=api`.

Explication. Dès qu'une politique sélectionne `app=api`, seul ce qu'elle liste entre. k3s applique les politiques (kube-router) ; le même YAML sur un CNI qui les ignore ne changerait rien, et seul ce test le révèle.

Nettoyage

```
kubectl delete namespace lab16
kubectl config set-context --current --namespace=default
```

Le namespace emporte Deployment, Service, Pods, LimitRange, quota, ServiceAccount, Role, RoleBinding et NetworkPolicy. L'image `api:1.0` reste dans containerd pour le labo 17.

Ce que vous devez pouvoir affirmer maintenant

- `Allocated resources` compte les requests ; `kubectl top` mesure le réel ; QoS `Burstable` quand requests ≠ limits.
- `OOMKilled`, `Exit Code 137`, `RESTARTS` qui monte : la limit mémoire ; `Pending` + `Insufficient memory` : la request.
- LimitRange : défauts injectés dans un Pod sans `resources`, refus au-delà de `max` ; quota : `Used` / `Hard` par namespace.
- `logs --previous` pour le conteneur mort, `debug --target` pour un shell à côté d'une image sans shell.
- `auth can-i … --as=system:serviceaccount:ns:nom` : yes/no, et `Forbidden` en vrai.
- NetworkPolicy : `Connection refused` sans le label, `UP` avec ; appliquée par le CNI de k3s.